

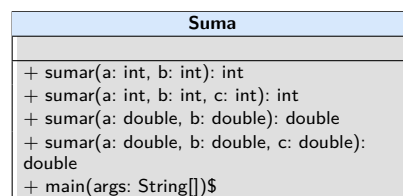
Entrega Actividad 6: Ejercicios Propuestos

1. Ejercicio 2.10. Sobrecarga de métodos

Enunciado: Definir una clase denominada Suma, la cual tiene varios métodos sumar sobrecargados:

- Un método sumar que obtiene la suma de dos valores enteros pasados como parámetros.
- Un método sumar que obtiene la suma de tres valores enteros pasados como parámetros.
- Un método sumar que obtiene la suma de dos valores double pasados como parámetros.
- Un método sumar que obtiene la suma de tres valores double pasados como parámetros.

1.1. Diagrama de clases



1.2. Código Fuente

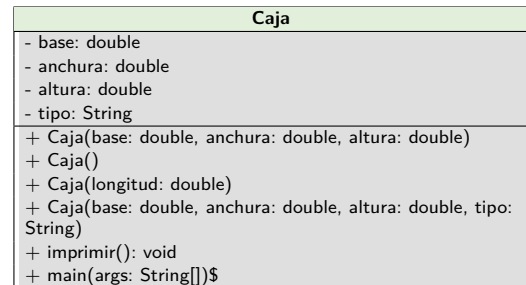
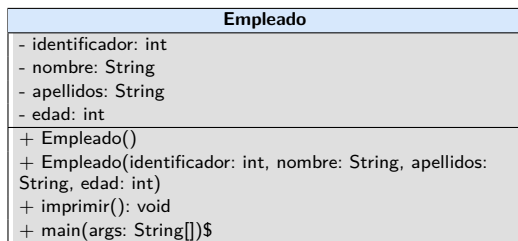
```
1 package Ejercicio_2_10;
2
3 public class Suma {
4     public int sumar(int a, int b) {
5         return a + b;
6     }
7
8     public int sumar(int a, int b, int c) {
9         return a + b + c;
10    }
11
12    public double sumar(double a, double b) {
13        return a + b;
14    }
15
16    public double sumar(double a, double b, double c) {
17        return a + b + c;
18    }
19
20    public static void main(String[] args) {
21        Suma suma = new Suma();
22        System.out.println("Suma_2_ints_(2+3):_" + suma.sumar(2, 3));
23        System.out.println("Suma_3_ints_(2+3+4):_" + suma.sumar(2, 3, 4));
24        System.out.println("Suma_2_doubles_(2.5+3.5):_" + suma.sumar(2.5, 3.5));
25        System.out.println("Suma_3_doubles_(2.5+3.5+4.5):_" + suma.sumar(2.5, 3.5, 4.5));
26    }
27 }
```

2. Ejercicio 2.11. Sobrecarga de constructores

Enunciado:

- Definir una clase Empleado que tiene como atributos: identificador, nombre, apellidos y edad. Contiene dos constructores: uno sin parámetros con valores por defecto y otro con parámetros.
- Definir una clase Caja que tiene como atributos base, anchura, altura y tipo. Contiene varios constructores sobrecargados, incluyendo uno que invoca a otro mediante `this()`.

2.1. Diagrama de clases



2.2. Código Fuente

Clase Empleado:

```
1 package Ejercicio_2_11;
2
3 public class Empleado {
4     private int identificador;
5     private String nombre;
6     private String apellidos;
7     private int edad;
8
9     public Empleado() {
10         this.identificador = 100;
11         this.nombre = "Nuevo_Empleado";
12         this.apellidos = "Nuevo_Apellidos";
13         this.edad = 18;
14     }
15
16     public Empleado(int identificador, String nombre, String apellidos, int edad) {
17         this.identificador = identificador;
18         this.nombre = nombre;
19         this.apellidos = apellidos;
20         this.edad = edad;
21     }
22
23     public void imprimir() {
24         System.out.println("ID: " + identificador + ", Nombre: " + nombre + ", Apellidos: " + apellidos + ", Edad: " + edad);
25     }
26
27     public static void main(String[] args) {
28         Empleado emp1 = new Empleado();
29         Empleado emp2 = new Empleado(101, "Juan", "Perez", 30);
30         emp1.imprimir();
31         emp2.imprimir();
32     }
33 }
```

Clase Caja:

```
1 package Ejercicio_2_11;
2
3 public class Caja {
4     private double base;
```

```

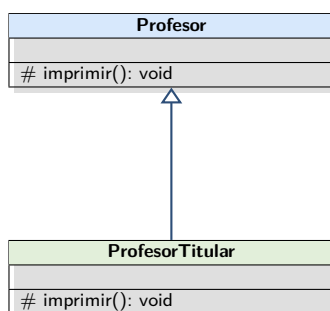
5     private double anchura;
6     private double altura;
7     private String tipo;
8
9     public Caja(double base, double anchura, double altura) {
10         this.base = base;
11         this.anchura = anchura;
12         this.altura = altura;
13         this.tipo = "Desconocido";
14     }
15
16     public Caja() {
17         this.base = 0;
18         this.anchura = 0;
19         this.altura = 0;
20         this.tipo = "Desconocido";
21     }
22
23     public Caja(double longitud) {
24         this.base = longitud;
25         this.anchura = longitud;
26         this.altura = longitud;
27         this.tipo = "Desconocido";
28     }
29
30     public Caja(double base, double anchura, double altura, String tipo) {
31         this(base, anchura, altura);
32         this.tipo = tipo;
33     }
34
35     public void imprimir() {
36         System.out.println("Base:␣" + base + ",␣Anchura:␣" + anchura + ",␣Altura:␣" + altura +
37     }
38
39     public static void main(String[] args) {
40         Caja caja1 = new Caja(10.5, 20.0, 15.5);
41         Caja caja2 = new Caja();
42         Caja caja3 = new Caja(12.0);
43         Caja caja4 = new Caja(10.0, 10.0, 10.0, "Cubo");
44
45         caja1.imprimir();
46         caja2.imprimir();
47         caja3.imprimir();
48         caja4.imprimir();
49     }
50 }

```

3. Ejercicio 4.4. Polimorfismo

Enunciado: ¿Cuál es el resultado de la ejecución del siguiente programa basado en el ejercicio anterior (donde se hace un casting de la instancia de la subclase a la superclase)?

3.1. Diagrama de clases



3.2. Código Fuente y Resultado

Clase Profesor y ProfesorTitular:

```
1 package Profesores;
2
3 public class Profesor {
4     protected void imprimir() {
5         System.out.println("Es un profesor.");
6     }
7 }
```

```
1 package Profesores;
2
3 public class ProfesorTitular extends Profesor {
4     protected void imprimir() {
5         System.out.println("Es un profesor titular.");
6     }
7 }
```

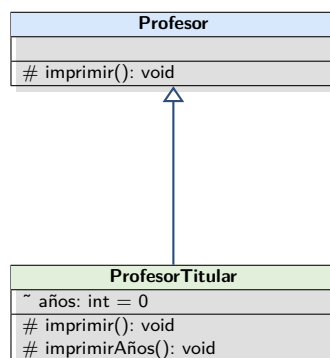
Clase Prueba:

```
1 package Profesores;
2
3 public class Prueba {
4     public static void main(String[] args) {
5         Profesor profesor1 = new ProfesorTitular();
6         Profesor profesor2 = (Profesor) profesor1;
7         profesor2.imprimir();
8
9         // El resultado de la ejecución es "Es un profesor titular." debido al polimorfismo,
10        // ya que el objeto subyacente instanciado en memoria es un ProfesorTitular.
11        // A pesar del casting, en tiempo de ejecución se invoca el método sobrescrito de la s
12    }
13 }
```

4. Ejercicio 4.6. Métodos polimórficos

Enunciado: ¿Cuál es el resultado de la ejecución del siguiente programa que utiliza un Vector de objetos Profesor instanciados tanto como Profesor y como ProfesorTitular?

4.1. Diagrama de clases



4.2. Código Fuente y Resultado

Clase Prueba:

```
1 package Profesores;
2 import java.util.*;
3
4 public class Prueba {
5     Vector<Profesor> profesores;
```

```

6
7 public static void main(String[] args) {
8     Prueba prueba = new Prueba();
9     prueba.profesores = new Vector<Profesor>();
10    Profesor profesor1 = new Profesor();
11    ProfesorTitular profesor2 = new ProfesorTitular();
12
13    prueba.profesores.add(profesor1);
14    prueba.profesores.add(profesor2);
15
16    for(int i = 0; i < prueba.profesores.size(); i++) {
17        Profesor p = (Profesor) prueba.profesores.elementAt(i);
18        p.imprimir();
19    }
20
21    // Resultado esperado:
22    // Es un profesor.
23    // Es un profesor titular.
24
25    // Explicación:
26    // El bucle recorre el vector que contiene elementos del tipo base 'Profesor'.
27    // En la primera iteración, 'p' hace referencia a un objeto 'Profesor', llamando a su método imprimir().
28    // En la segunda iteración, 'p' hace referencia a un objeto 'ProfesorTitular'. Debido a la herencia,
29    // al invocar 'imprimir()' se ejecuta la versión redefinida en la subclase, a pesar de que el objeto
30    // sea de tipo 'Profesor'.
31 }
32 }

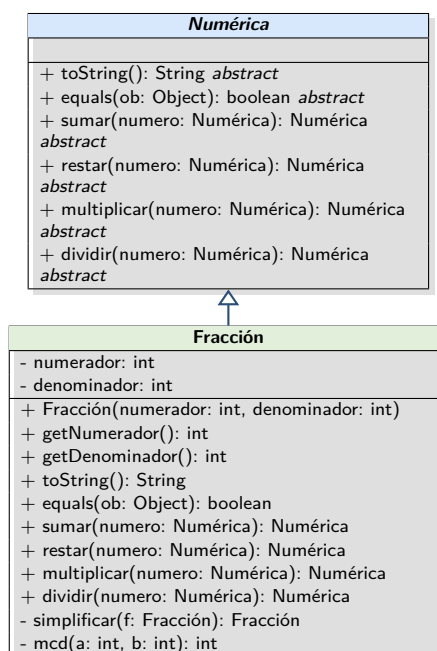
```

5. Ejercicio 4.7. Clases abstractas

Enunciado:

- Definir una clase abstracta denominada Numérica con métodos abstractos: toString(), equals(), sumar(), restar(), multiplicar() y dividir().
- Definir una clase Fracción que representa un número fraccionario, hereda de Numérica, implementa sus métodos y tiene numerador y denominador.
- Crear una clase de prueba que utilice los métodos implementados.

5.1. Diagrama de clases



5.2. Código Fuente

Clase Numérica:

```
1 package Ejercicio_4_7;
2
3 public abstract class Numérica {
4     public abstract String toString();
5     public abstract boolean equals(Object ob);
6     public abstract Numérica sumar(Numérica número);
7     public abstract Numérica restar(Numérica número);
8     public abstract Numérica multiplicar(Numérica número);
9     public abstract Numérica dividir(Numérica número);
10 }
```

Clase Fracción:

```
1 package Ejercicio_4_7;
2
3 public class Fracción extends Numérica {
4     private int numerador;
5     private int denominador;
6
7     public Fracción(int numerador, int denominador) {
8         this.numerador = numerador;
9         this.denominador = denominador != 0 ? denominador : 1; // Prevenir división por 0
10    }
11
12    public int getNumerador() { return numerador; }
13    public int getDenominador() { return denominador; }
14
15    @Override
16    public String toString() {
17        return numerador + "/" + denominador;
18    }
19
20    @Override
21    public boolean equals(Object ob) {
22        if (ob instanceof Fracción) {
23            Fracción f = (Fracción) ob;
24            // Comparación de fracciones usando producto cruzado
25            return this.numerador * f.denominador == this.denominador * f.numerador;
26        }
27        return false;
28    }
29
30    @Override
31    public Numérica sumar(Numérica número) {
32        if (número instanceof Fracción) {
33            Fracción f = (Fracción) número;
34            int nuevoNumerador = this.numerador * f.denominador + f.numerador * this.denominador;
35            int nuevoDenominador = this.denominador * f.denominador;
36            return simplificar(new Fracción(nuevoNumerador, nuevoDenominador));
37        }
38        return null;
39    }
40
41    @Override
42    public Numérica restar(Numérica número) {
43        if (número instanceof Fracción) {
44            Fracción f = (Fracción) número;
45            int nuevoNumerador = this.numerador * f.denominador - f.numerador * this.denominador;
46            int nuevoDenominador = this.denominador * f.denominador;
47            return simplificar(new Fracción(nuevoNumerador, nuevoDenominador));
48        }
49        return null;
50    }
51 }
```

```

52     @Override
53     public Numérica multiplicar(Numérica número) {
54         if (número instanceof Fracción) {
55             Fracción f = (Fracción) número;
56             int nuevoNumerador = this.numerador * f.numerador;
57             int nuevoDenominador = this.denominador * f.denominador;
58             return simplificar(new Fracción(nuevoNumerador, nuevoDenominador));
59         }
60         return null;
61     }
62
63     @Override
64     public Numérica dividir(Numérica número) {
65         if (número instanceof Fracción) {
66             Fracción f = (Fracción) número;
67             int nuevoNumerador = this.numerador * f.denominador;
68             int nuevoDenominador = this.denominador * f.numerador;
69             return simplificar(new Fracción(nuevoNumerador, nuevoDenominador));
70         }
71         return null;
72     }
73
74     private Fracción simplificar(Fracción f) {
75         int mcd = mcd(Math.abs(f.getNumerador()), Math.abs(f.getDenominador()));
76         return new Fracción(f.getNumerador() / mcd, f.getDenominador() / mcd);
77     }
78
79     private int mcd(int a, int b) {
80         if (b == 0) {
81             return a;
82         }
83         return mcd(b, a % b);
84     }
85 }

```

Clase Prueba:

```

1  package Ejercicio_4_7;
2
3  public class Prueba {
4      public static void main(String[] args) {
5          Fracción f1 = new Fracción(1, 2);
6          Fracción f2 = new Fracción(3, 4);
7
8          System.out.println("Fracción 1: " + f1.toString());
9          System.out.println("Fracción 2: " + f2.toString());
10
11         System.out.println("¿Son iguales? " + f1.equals(f2));
12
13         Numérica suma = f1.sumar(f2);
14         System.out.println("Suma (1/2 + 3/4): " + suma.toString());
15
16         Numérica resta = f1.restar(f2);
17         System.out.println("Resta (1/2 - 3/4): " + resta.toString());
18
19         Numérica multiplicacion = f1.multiplicar(f2);
20         System.out.println("Multiplicación (1/2 * 3/4): " + multiplicacion.toString());
21
22         Numérica division = f1.dividir(f2);
23         System.out.println("División (1/2 / 3/4): " + division.toString());
24
25         // Otro caso de prueba (Fracciones iguales)
26         Fracción f3 = new Fracción(2, 4);
27         System.out.println("\n¿1/2 es igual a 2/4? " + f1.equals(f3));
28     }
29 }

```
